Einführung

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 15.10.2026

Organisatorisches

- **Personen**:
  - Dr. Fabian Scheipl (VL)
  - Jana Gauß (Ü)
  - Maxim Winter (Tutorium)
- **Zeiten & Räume**:

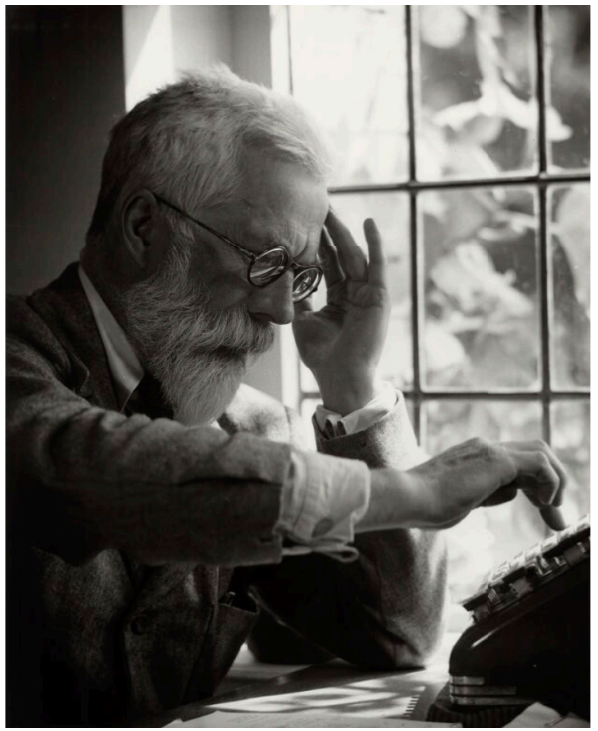
VL	Do 10:15-12:00	Hgb. – M 114
Übung 1	Di 14:15-15:45	Hgb. – M 105
Übung 2	Mi 12:15-13:45	Hgb. – B 106
Tutorium “Mathematische Grundlagen”	Di 10:00-12:00	S 003 (Schellingstr. 3)

- **Termine WS 26/27**: wöchentlich Do, **15.10.2026 – 4.2.2027** (Weihnachtspause: letzte VL 17.12.2026, weiter am 7.1.2027)

Motivation

Inhalt: Statistik früher & heute

Statistik früher



Statistik heute



Motivation

- In der Praxis ist Statistik / ML immer *computational*.
- In diesem Kurs lernen wir *mathematische Konzepte*, die bei der praktischen Lösung von Problemen helfen.
- Diese Konzepte werden benötigt, um *Computeralgorithmen* zu entwickeln, verstehen und analysieren.

  $\Longrightarrow$ fortgeschrittene (numerische) Lineare Algebra

  $\Longrightarrow$ fortgeschrittene Analysis

Warum so viel Mathe? Ein Beispiel

Aufgabe: Trainiere ein lineares Modell $y_i \approx \beta_1 x_{i1} + \cdots + \beta_p x_{ip}$ mit $p = 10.000$ Features auf $n = 100.000$ Beobachtungen.

$\Longrightarrow$ **Kleinste-Quadrate-Problem:**

$$\min_{\boldsymbol{\beta} \in \mathbb{R}^p} \left(\sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{p} x_{ij} \beta_j \right)^2 \right) = \min_{\boldsymbol{\beta} \in \mathbb{R}^p} \|\boldsymbol{X}\boldsymbol{\beta} - \boldsymbol{y}\|_2^2$$

Naive Lösung (falls $\operatorname{rang}(\boldsymbol{X}) = p$)**:** $\widehat{\boldsymbol{\beta}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$

Probleme in der Praxis:

1. **Kondition**: $\boldsymbol{X}^\top \boldsymbol{X}$ kann sehr schlecht “konditioniert” sein $\Longrightarrow$ numerische Instabilität, große Fehler
2. **Komplexität**: Matrixinversion kostet $O(p^3)$ Operationen $\Longrightarrow$ für $p = 10.000$: ca. 10^{12} FLOPs!
3. **Speicher**: $\boldsymbol{X}^\top \boldsymbol{X}$ ist $p \times p$ Matrix $\Longrightarrow$ für $p = 10.000$: ca. 800 MB nur für diese Matrix

Bessere Lösungen: QR-Zerlegung, SVD, iterative Verfahren $\rightarrow$ dieser Kurs

Themen

- Kondition und Komplexität
  - Was sind gute Algorithmen?
- Numerische Lineare Algebra
  - Matrix- & Tensorrechnung
  - Berechne (KQ-)Lösungen für $\boldsymbol{Ax} \approx \boldsymbol{b}$ etc.
- Fortgeschrittene Differenzialrechnung / Analysis
  - Höherdimensionale, verallgemeinerte Ableitungen
  - Wie kann man komplizierte Fkt. einfach approximieren?
- Numerische Optimierung
  - Wie finde ich Minima/Maxima von komplizierten Funktionen?
  - z.B. Training neuronaler Netze: Millionen Parameter, nicht-konvexe Verlustfunktionen $\rightarrow$ smarte Algorithmen nötig (SGD & Co.)

Fünf Beispiele

Beispiel 1: Kondition

Betrachte das lineare Gleichungssystem $\boldsymbol{Ax} = \boldsymbol{b}$ mit

$$\boldsymbol{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1.0001 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 2 \\ 2.0001 \end{pmatrix}$$

Exakte Lösung: $\boldsymbol{x} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

Gestörtes Problem (z.B. durch Rundungsfehler): $\boldsymbol{Ax} = \tilde{\boldsymbol{b}}$ mit

$$\tilde{\boldsymbol{b}} = \begin{pmatrix} 2 \\ 2.0002 \end{pmatrix} \quad (\text{nur } 0.005\% \text{ Fehler!})$$

Lösung des gestörten Problems: $\tilde{\boldsymbol{x}} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$

$\Longrightarrow$ **100% Fehler** in x_1 durch 0.005% Fehler in den Daten!

$\Longrightarrow$ Das Problem ist (furchtbar) **schlecht konditioniert**.

Beispiel 2: Komplexität

Matrixmultiplikation $C = AB$ für $n \times n$ Matrizen:

Algorithmus	**Rechenops. für $n = 1000$**	**Komplexität**
Naive Methode	$\approx 10^9$ Ops	$O(n^3)$
Strassen	$\approx 6 \times 10^8$ Ops	$O(n^{2.807})$
Optimiert	$\approx 2 \times 10^8$ Ops	$O(n^{2.373})$

Für $n = 10.000$:

- Naiv: $\approx 10^{12}$ Ops
- Optimiert: $\approx 2 \times 10^{10}$ Ops

$\Longrightarrow$ **Faktor 50 schneller** durch besseren Algorithmus (...potentiell)!

Beispiel 3: Höherdimensionale Ableitungen

Beispiel AI: Gradient Descent für neuronale Netze

Verlustfunktion: $L : \mathbb{R}^{p \times K} \to \mathbb{R}$ (Matrix-Argument $\boldsymbol{W}$!)

Zum Beispiel: **MSE**

$$L(\boldsymbol{W}) = \frac{1}{2} \|\boldsymbol{X}\boldsymbol{W} - \boldsymbol{Y}\|_F^2$$

mit $\boldsymbol{X} \in \mathbb{R}^{N \times p}$, $\boldsymbol{W} \in \mathbb{R}^{p \times K}$, $\boldsymbol{Y} \in \mathbb{R}^{N \times K}$

Frage: Wie berechnen wir $\frac{\partial L}{\partial \boldsymbol{W}}$?

Naive Methode: Berechne $\frac{\partial L}{\partial W_{ij}}$ für alle i, j einzeln

$\implies$ $p \times K$ Ableitungen!

Für $p = 1000, K = 10$: **10.000 Ableitungen**!

Bessere Methode: Matrix-Kalkül

$$\frac{\partial L}{\partial \boldsymbol{W}} = \boldsymbol{X}^\top (\boldsymbol{X}\boldsymbol{W} - \boldsymbol{Y})$$

Eine Zeile statt 10.000! Und: **effizient implementierbar**.

$\implies$ Ableitungen von Matrix/Tensor-Funktionen essentiell für Deep Learning, Optimierung, etc.

Beispiel 4: Funktionsapproximation: Taylor-Reihen

Ziel: Approximiere komplizierte Funktion $f(x)$ durch ein Polynom in der Nähe von x_0

Taylor-Approximation 2. Ordnung:

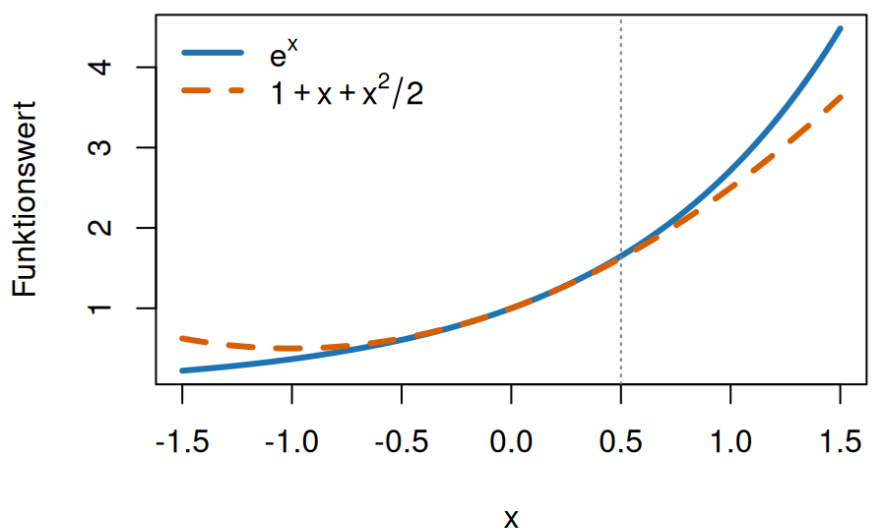
$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2}(x - x_0)^2$$

Konkretes Beispiel: $f(x) = e^x$ um $x_0 = 0$

$$f(0) = f'(0) = f''(0) = 1 \implies e^x \approx 1 + x + \frac{x^2}{2}$$

Fehler für $x = 0.5$:

- Exakt: $e^{0.5} = 1.6487\ldots$
- Approximation: $1 + 0.5 + 0.125 = 1.625$
- Relativer Fehler: $< 2\%$



Beispiel 5: Optimierung in Machine Learning

Typisches Problem: Training eines neuronalen Netzes

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} L(\boldsymbol{\theta}), \qquad L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(\boldsymbol{x}_i; \boldsymbol{\theta}), y_i)$$

Modell	**Parameter** d	**Daten** N
Logistische Regression (MNIST, 10 Klassen)	$\sim 8 \times 10^3$	6×10^4
ResNet-50	$\sim 2.5 \times 10^7$	$> 10^6$
GPT-3	$\sim 1.75 \times 10^{11}$	$\sim 3 \times 10^{11}$ Tokens

Herausforderungen:

1. **Nicht-konvex:** Viele lokale Minima / Sattelpunkte der Verlustfunktion
2. **Hochdimensional:** Naive Methoden scheitern
3. **Stochastisch:** Nur Zugriff auf Minibatches

$\Longrightarrow$ Brauchen **smarte Algorithmen**: Stochastic Gradient Descent, Momentum, etc.

Vorkenntnisse & Selbsttest

Benötigte Vorkenntnisse

- Lineare Algebra (VL Lineare Algebra I):
  - v.a. Matrixrechnung, Eigenwerte und -vektoren
- Analysis (VL Analysis I):
  - v.a. Stetigkeit & Differenzierbarkeit, Ableitungen
- R-Programmierung (VL Einführung in R)

Empfehlung:

- Intuition für LA z.B. mit *3blue1brown* o.ä. aufbauen!
- Skript **aktiv** lesen & benutzen um Grundbegriffe zu wiederholen

(Minimaler) Selbsttest: Vorkenntnisse evtl. ausreichend?

Lineare Algebra - Sie sollten wissen:

1. Was ist der *Rang* einer Matrix? Wie berechne ich ihn?
2. Was bedeutet es, wenn eine Matrix $\boldsymbol{A}$ *invertierbar* ist?
3. Was sind *Eigenwerte*/*Eigenvektoren* einer Matrix? Wie finde ich sie?
4. Was ist die geometrische Bedeutung einer Matrix-Vektor-Multiplikation $\boldsymbol{Ax}$?
5. Wann bilden Vektoren $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$ eine *Basis* des $\mathbb{R}^n$? Was ist ihr *Span*?

(Minimaler) Selbsttest: Vorkenntnisse evtl. ausreichend?

Analysis - Sie sollten wissen:

1. Was bedeutet es, wenn eine Funktion $f : \mathbb{R} \to \mathbb{R}$ *stetig* ist?
2. Was ist die *Ableitung* $f'(x_0)$ geometrisch?
3. Was ist ein lokales vs. globales Minimum?